

LAPORAN PRAKTEK
Pemrograman Struktur Data
(Basis Data NoSQL)



DHIA ATHALLA (225443006)



TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI
Politeknik Manufaktur Bandung
Tahun Ajaran 2025/2026

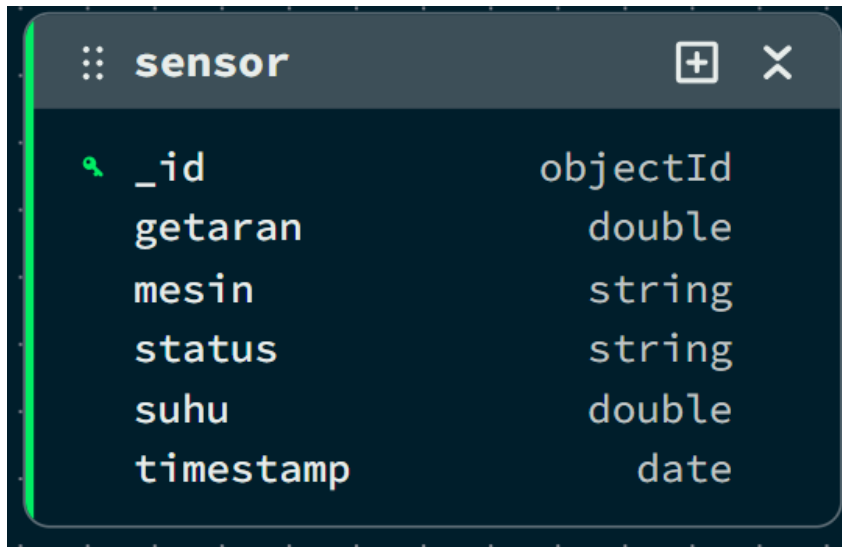
PENDAHULUAN

Dalam praktikum ini, kita menggunakan konsep *aggregation pipeline*. Konsep *Aggregation framework* adalah alat utama di MongoDB untuk memproses, mengubah, dan menganalisis data. Data mengalir melalui serangkaian *stage*, di mana output dari satu stage menjadi input stage berikutnya. Tujuan mempelajari serangkaian materi ini agar dapat menjelaskan konsep aggregation pipeline dan cara kerjanya serta mampu menggunakan stage-stage utama: `$match`, `$group`, `$sort`, `$project`, `$unwind`, `$lookup`, `$addFields`, `$bucket`, `$facet` dan mampu menerapkan akumulator (`$sum`, `$avg`, `$min`, `$max`, `$push`, `$addToSet`, `$first`, `$last`) untuk menghasilkan ringkasan data industri.

Selain itu, kita juga bisa memanfaatkan operator ekspresi (`$dateToString`, `$hour`, `$cond`, `$ifNull`, `$multiply`, `$divide`) untuk transformasi data di dalam pipeline. Serta menggabungkan data dari beberapa koleksi menggunakan `$lookup` (join) dan merancang pipeline agregasi untuk menjawab pertanyaan bisnis nyata: total produksi per mesin, rata-rata suhu per jam, reject rate, OEE, distribusi nilai sensor. Dengan hal itu kita bisa mengoptimasi pipeline dengan menempatkan `$match` dan `$limit` di awal, serta memproyeksikan field yang diperlukan saja.

DESAIN DATA

PRAKTIKUM TERBIMBING



The image shows a screenshot of a MongoDB database interface. At the top, there is a tab labeled 'sensor' with a search icon on the left and add and close icons on the right. Below the tab, a table displays the schema of the 'sensor' collection. The table has two columns: the field name and its data type. The fields listed are '_id' (objectId), 'getaran' (double), 'mesin' (string), 'status' (string), 'suhu' (double), and 'timestamp' (date). A green cursor is visible on the left side of the table.

Field	Type
_id	objectId
getaran	double
mesin	string
status	string
suhu	double
timestamp	date

PIPELINE DAN HASIL

1. Kode Pipeline

PRAKTIKUM TERBIMBING

- Koneksi.py

```
import os
from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"), serverSelectionTimeoutMS=5000)

try:
    client.admin.command('ping')
    print("Koneksi MongoDB berhasil!")
except Exception as e:
    print("Gagal terhubung:", e)
finally:
    client.close()
```

menghubungkan koneksi dari MongoDB menggunakan kode python dan melakukan PING! Untuk mengecek apakah koneksinya berhasil atau tidak

- Crud Sensor.py

```
# import
from dotenv import load_dotenv
from pymongo import MongoClient
from datetime import datetime, timedelta
import random, os

# module
load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]

# Insert data sensor acak
data = []
for i in range(10):
    doc = {
        "mesin": f"CNC-{random.randint(1,3):02d}",
        "suhu": round(random.uniform(60, 100), 2),
        "getaran": round(random.uniform(0.1, 0.5), 2),
        "timestamp": datetime.utcnow() - timedelta(minutes=i*5),
        "status": "normal"
    }
```

```

        data.append(doc)
result = collection.insert_many(data)
print(f"Tersimpan {len(result.inserted_ids)} dokumen")

# Baca data CNC-01 terbaru
cursor = collection.find(
    {"mesin": "CNC-01"},
    {"_id": 0, "suhu": 1, "timestamp": 1}
).sort("timestamp", -1).limit(5)
print("Data CNC-01 terbaru:")
for doc in cursor:
    print(doc)

# Update status menjadi "maintenance" jika suhu > 90
update_result = collection.update_many(
    {"suhu": {"$gt": 90}},
    {"$set": {"status": "maintenance"}}
)
print(f"Update: {update_result.modified_count} dokumen diubah")

# Hapus data yang lebih lama dari 1 jam
# satu_jam_lalu = datetime.utcnow() - timedelta(hours=1)
# delete_result = collection.delete_many({"timestamp": {"$lt":
satu_jam_lalu}})
# print(f>Delete: {delete_result.deleted_count} dokumen dihapus")

import logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[
        logging.FileHandler("crud_sensor.log"),
        logging.StreamHandler()
    ]
)
try:
    result = collection.insert_many(data)
    logging.info(f"Insert berhasil, {len(result.inserted_ids)} dokumen")
except Exception as e:
    logging.error(f"Insert gagal: {e}")

```

Di dalam kode ini, data akan dibuat dalam bentuk acak, jadi datanya akan berbeda setiap orang yang menggunakan kode ini. Kita menggunakan *insert_many* untuk efisiensi; 10 data dimasukkan dalam satu perintah. Loop for i menciptakan timestamp yang mundur 5 menit setiap langkah agar data tidak semuanya pada waktu yang sama, lebih realistis. *random.randint* dan *random.uniform* memberi variasi mesin dan nilai sensor. Dan ada fitur CRUD di dalam kode nya juga.

- Agregat.py

```
import os
```

```

from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]

pipeline = [
    {"$group": {
        "_id": "$mesin",
        "rata_suhu": {"$avg": "$suhu"},
        "max_getaran": {"$max": "$getaran"},
        "count": {"$sum": 1}
    }},
    {"$sort": {"rata_suhu": -1}}
]

results = collection.aggregate(pipeline)
print("Rata-rata suhu per mesin:")
for doc in results:
    print(f"{doc['_id']} -> rata:{doc['rata_suhu']:.2f}°C, max  
getaran:{doc['max_getaran']}, jumlah data:{doc['count']}")

client.close()

```

kode ini akan melakukan beberapa stage untuk mengolah data dan melakukan pengeluaran data yang telah dilakukan di dalam pipeline.

- Analisis.py

```

import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import os
from dotenv import load_dotenv
from pymongo import MongoClient

# module
load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]

# ambil data sensor
cursor = collection.find({}, {"_id": 0})
df = pd.DataFrame(list(cursor))

```

```

df['timestamp'] = pd.to_datetime(df['timestamp'])
# 1. Jadikan timestamp sebagai index
df.set_index('timestamp', inplace=True)

# 2. Urutin index (waktu) dari lama ke baru
df.sort_index(inplace=True)

resampled = df.resample('10min').mean(numeric_only=True)
print(resampled.head())

plt.figure(figsize=(10,5))

# Plotting datanya
df['suhu'].plot(title='Suhu dari Waktu ke Waktu')

plt.ylabel('Suhu (°C)')
plt.grid(True)
plt.savefig('suhu_plot1.png')
plt.show()

resampled.to_csv('agregasi.csv')

```

pandas akan melakukan operasi untuk menyusun data yang telah di bangun sebelumnya. Hati hati dalam mengelola data yang akan di tampilkan ke dalam sebuah grafik, terkadang grafik akan terlihat seperti benang kusut jika tidak di urutkan datanya.

- Data generator.py

```

import time, random
from datetime import datetime
from pymongo import MongoClient
import os
from dotenv import load_dotenv

load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]

while True:
    doc = {
        "mesin": f"CNC-{random.randint(1,5):02d}",
        "suhu": round(random.uniform(60, 100), 2),
        "getaran": round(random.uniform(0.1, 0.5), 2),
        "timestamp": datetime.utcnow()
    }
    collection.insert_one(doc)
    print(f"[{datetime.utcnow()}] Inserted: {doc['mesin']} - {doc['suhu']}°C")
    time.sleep(2)

```

kode ini akan menghasilkan data yang akan terus menghasilkan data dummy yang berhenti jika kita memberhentikan loop nya dari terminal.

- Mqtt_publisher dan mqtt_subscriber

Publisher kode

```
import json, time, random
import paho.mqtt.client as mqtt
from datetime import datetime

BROKER = "broker.hivemq.com"
PORT = 1883
TOPIC = "pabrik/sensor/suhu"

client = mqtt.Client()
client.connect(BROKER, PORT, 120)
client.loop_start()

while True:
    data = {
        "mesin": f"CNC-{random.randint(1,5):02d}",
        "suhu": round(random.uniform(60, 100), 2),
        "getaran": round(random.uniform(0.1, 0.5), 2),
        "timestamp": datetime.utcnow().isoformat()
    }
    payload = json.dumps(data)
    client.publish(TOPIC, payload)
    print(f"[PUB] {payload}")
    time.sleep(2)
```

Subscriber kode

```
import json, os
import paho.mqtt.client as mqtt
from pymongo import MongoClient
from datetime import datetime
from dotenv import load_dotenv

load_dotenv()
MONGO_URI = os.getenv("MONGO_URI")
DB_NAME = os.getenv("DB_NAME")

print(f"Using DB: {DB_NAME}")
mongo_client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000)
db = mongo_client[DB_NAME]
collection = db["sensor"]
print("MongoDB ready - collection 'sensor'")
```



```

def on_connect(client, userdata, flags, rc):
    print(f"MQTT Connected with code {rc}")
    if rc == 0:
        client.subscribe("pabrik/sensor/suhu")
        print("✓ Subscribed to pabrik/sensor/suhu")
    else:
        print(f"X Connection failed: {rc}")

def on_message(client, userdata, msg):
    try:
        print(f"Message on {msg.topic}: {msg.payload.decode()}")
        payload = json.loads(msg.payload.decode())
        payload['timestamp'] = datetime.fromisoformat(payload['timestamp'])
        result = collection.insert_one(payload)
        print(f"✓ Saved to MongoDB ID: {result.inserted_id} | {payload['mesin']} {payload['suhu']}°C")
    except Exception as e:
        print(f"X Message error: {e}")

mqtt_client = mqtt.Client()
mqtt_client.on_connect = on_connect
mqtt_client.on_message = on_message

print("Connecting to broker.hivemq.com (120s timeout)...")
mqtt_client.connect("broker.hivemq.com", 1883, 120)
mqtt_client.loop_forever()

```

- Logging.py

```

# import
from dotenv import load_dotenv
from pymongo import MongoClient
from datetime import datetime, timedelta
import random, os

# module
load_dotenv()
client = MongoClient(os.getenv("MONGO_URI"))
db = client[os.getenv("DB_NAME")]
collection = db["sensor"]

# logging
import logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',

```

```

handlers=[
    logging.FileHandler("crud_sensor.log"),
    logging.StreamHandler()
]
)
try:
    result = collection.insert_many(data)
    logging.info(f"Insert berhasil, {len(result.inserted_ids)} dokumen")
except Exception as e:
    logging.error(f"Insert gagal: {e}")

# Insert data sensor acak
data = []
for i in range(10):
    doc = {
        "mesin": f"CNC-{random.randint(1,3):02d}",
        "suhu": round(random.uniform(60, 100), 2),
        "getaran": round(random.uniform(0.1, 0.5), 2),
        "timestamp": datetime.utcnow() - timedelta(minutes=i*5),
        "status": "normal"
    }
    data.append(doc)
result = collection.insert_many(data)
print(f"Tersimpan {len(result.inserted_ids)} dokumen")

# Baca data CNC-01 terbaru
cursor = collection.find(
    {"mesin": "CNC-01"},
    {"_id": 0, "suhu": 1, "timestamp": 1}
).sort("timestamp", -1).limit(5)
print("Data CNC-01 terbaru:")
for doc in cursor:
    print(doc)

# Update status menjadi "maintenance" jika suhu > 90
update_result = collection.update_many(
    {"suhu": {"$gt": 90}},
    {"$set": {"status": "maintenance"}}
)
print(f"Update: {update_result.modified_count} dokumen diubah")

# Hapus data yang lebih lama dari 1 jam
# satu_jam_lalu = datetime.utcnow() - timedelta(hours=1)
# delete_result = collection.delete_many({"timestamp": {"$lt":
satu_jam_lalu}})
# print(f>Delete: {delete_result.deleted_count} dokumen dihapus")

```

a. PANDUAN TUGAS MANDIRI PERTEMUAN 5

- **Buat database baru dan buat file CSV**

```
mesin,tanggal,biaya,teknisi
Mesin_A01,2026-01-15,450000,Budi Santoso
Mesin_B02,2026-01-22,320000,Ahmad Wijaya
Mesin_A01,2026-02-03,280000,Siti Nurhaliza
Mesin_C03,2026-02-14,550000,Roni Hermawan
Mesin_B02,2026-03-01,420000,Dewi Lestari
Mesin_D04,2026-03-18,375000,Hendra Gunawan
Mesin_A01,2026-04-05,290000,Maya Kusuma
Mesin_C03,2026-04-22,480000,Budi Santoso
Mesin_B02,2026-05-02,350000,Ahmad Wijaya
Mesin_D04,2026-05-15,425000,Siti Nurhaliza
```

- **Script python buat import maintenance**

```
import pandas as pd
from pymongo import MongoClient

# Baca file CSV menggunakan pandas
df = pd.read_csv('maintenance.csv')

# Konversi kolom tanggal menjadi tipe datetime
df['tanggal'] = pd.to_datetime(df['tanggal'])

# Hubungkan ke MongoDB
client = MongoClient('mongodb://localhost:27017/')
db = client['latihan6']
collection = db['maintenance']

# Konversi DataFrame menjadi list of dictionary
data_list = df.to_dict(orient='records')

# Sisipkan seluruh data ke dalam koleksi menggunakan insert_many
result = collection.insert_many(data_list)

print(f"Data berhasil diinsert. Total {len(result.inserted_ids)} dokumen ditambahkan.")
print(f"ID dokumen yang diinsert: {result.inserted_ids}")
```

- Query maintenance

```
import pandas as pd
from pymongo import MongoClient
from datetime import datetime

client = MongoClient('mongodb://localhost:27017/')
db = client['latihan6']
collection = db['maintenance']

print("=" * 70)
print("1. MENCARI DOKUMEN DENGAN BIAYA LEBIH DARI 1.000.000")
print("=" * 70)

# Cari dokumen dengan biaya > 1.000.000
query_high_cost = collection.find({"biaya": {"$gt": 1000000}})
high_cost_data = list(query_high_cost)

if high_cost_data:
    df_high_cost = pd.DataFrame(high_cost_data)
    # Hapus kolom _id jika ingin tampilan lebih rapi
    if '_id' in df_high_cost.columns:
        df_high_cost = df_high_cost.drop('_id', axis=1)
    print(f"\nDitemukan {len(high_cost_data)} dokumen:")
    print(df_high_cost.to_string(index=False))
else:
    print("\nTidak ada dokumen dengan biaya > 1.000.000")

print("\n" + "=" * 70)
print("2. UPDATE DOKUMEN DENGAN MESIN = 'CNC-01' DAN BIAYA = 1200000")
print("=" * 70)

# Update dokumen
update_result = collection.update_many(
    {"mesin": "CNC-01", "biaya": 1200000},
    {"$set": {"teknisi": "Dewi"}}
)

print(f"\nDokumen yang cocok: {update_result.matched_count}")
print(f"Dokumen yang diupdate: {update_result.modified_count}")

print("\n" + "=" * 70)
print("3. MENGHITUNG TOTAL BIAYA PER BULAN (AGGREGATION PIPELINE)")
print("=" * 70)

# Aggregation pipeline untuk total biaya per bulan
pipeline = [
    {
```

```

        "$group": {
            "_id": {
                "$dateToString": {
                    "format": "%Y-%m",
                    "date": "$tanggal"
                }
            },
            "total_biaya": {"$sum": "$biaya"}
        }
    },
    {
        "$sort": {"_id": 1}
    }
]

# Eksekusi aggregation
result_aggregation = list(collection.aggregate(pipeline))

if result_aggregation:
    df_monthly = pd.DataFrame(result_aggregation)
    df_monthly.columns = ['Bulan', 'Total Biaya']
    print("\nTotal Biaya per Bulan:")
    print(df_monthly.to_string(index=False))
    print(f"\nTotal keseluruhan: Rp {df_monthly['Total Biaya'].sum():,.0f}")
else:
    print("\nTidak ada data untuk aggregation")

client.close()
print("\n" + "=" * 70)
print("Selesai")
print("=" * 70)

```

- Publisher kode

```

import json
import random
import time
import paho.mqtt.client as mqtt

# --- KONFIGURASI ---
# Konfigurasi MQTT
broker = "170.106.137.233"
port = 34423
topic = "subpub"
username = "kell1"
password = "apaan"

```

```

client_id = f"pub-{{int(time.time())}}-{{random.randint(1000, 9999)}}"
client = mqtt.Client(client_id=client_id, clean_session=True)

batches = [f"BATCH-{{i:03d}}" for i in range(1, 21)]
mesins = ["CNC-01", "CNC-02", "LAS-01", "PRESS-01", "PAINT-01"]

def on_connect(client, userdata, flags, rc, properties=None):
    if rc == 0:
        print(f"✅ Publisher connected to MQTT broker (rc={{rc}})")
    else:
        print(f"❌ Connection failed, result code: {{rc}}")

def on_publish(client, userdata, mid):
    print(f"📩 Publish acknowledged (mid={{mid}})")

# =====
# WAJIB: Set username & password SEBELUM connect
# =====
client.username_pw_set(username, password)

client.on_connect = on_connect
client.on_publish = on_publish

print(f"Connecting to {{broker}}:{{port}} as client_id={{client_id}} ...")

try:
    client.connect(broker, port, keepalive=60)
    client.loop_start()

    while True:
        payload = {
            "batch": random.choice(batches),
            "mesin": random.choice(mesins),
            "jumlah": random.randint(100, 500),
            "reject": random.randint(0, 50),
        }

        # Pastikan reject tidak lebih dari jumlah
        if payload["reject"] > payload["jumlah"]:
            payload["reject"] = payload["jumlah"]

        payload_json = json.dumps(payload)

        # QoS 1 biar lebih pasti sampe
        info = client.publish(topic, payload_json, qos=1)

        if info.rc != mqtt.MQTT_ERR_SUCCESS:
            print(f"❌ Publish failed (rc={{info.rc}}) payload={{payload}}")

```

```

        else:
            print(f"Published to {topic}: {payload}")

            time.sleep(3)

except KeyboardInterrupt:
    print("Stopping publisher...")
finally:
    client.loop_stop()
    client.disconnect()

```

- Subscriber kode

```

import json
import time
import os
from datetime import datetime, timezone
import paho.mqtt.client as mqtt
from pymongo import MongoClient
from dotenv import load_dotenv

# Load env kalau lu mau simpen password di file .env (Sangat disarankan!)
load_dotenv()

# Konfigurasi MQTT
broker = "170.106.137.233"
port = 34423
topic = "subpub"
username = "kell"
password = "apaan"

# MongoDB
try:
    client_mongo = MongoClient('mongodb://localhost:27017/',
serverSelectionTimeoutMS=3000)
    client_mongo.admin.command('ping')
    db = client_mongo['mqtt_db']
    collection = db['produksi_mqtt']
    print("☑ MongoDB terhubung!")
except Exception as e:
    print(f"✗ MongoDB tidak terhubung. Error: {e}")
    collection = None

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print(f"☑ Berhasil konek ke broker (Result Code: {rc})")
        client.subscribe(topic)
    else:
        print(f"✗ Gagal konek, kode respon: {rc}")

```

```

def on_message(client, userdata, msg):
    print(f"Masuk: topic={msg.topic} payload={msg.payload!r}")
    try:
        payload = json.loads(msg.payload.decode())
    except json.JSONDecodeError:
        print("Bukan format JSON, diabaikan.")
        return

    # Pake timezone.utc biar sinkron sama database
    payload['timestamp'] = datetime.now(timezone.utc)

    # Kalkulasi Reject Rate buat monitoring produksi
    jumlah = payload.get('jumlah', 0)
    reject = payload.get('reject', 0)
    reject_rate = (reject / jumlah * 100) if jumlah else 0
    payload['reject_rate'] = round(reject_rate, 2)

    if reject_rate > 5:
        payload['peringatan'] = True
        print(f" Warning: Reject rate {payload['reject_rate']}% di
{payload.get('mesin')}")
    else:
        payload['peringatan'] = False

    # if collection:
    #     collection.insert_one(payload)
    #     print(f"💾 Tersimpan ke MongoDB")

# Inisialisasi Client
client = mqtt.Client()

# =====
# WAJIB: Pasang username & password SEBELUM connect()
# =====
client.username_pw_set(username, password)

client.on_connect = on_connect
client.on_message = on_message

print(f"Menghubungkan ke {broker}...")
client.connect(broker, port)

print(f"Subscriber listening to {topic}...")
client.loop_forever()

```


2. Tangkapan Layar Hasil
 - a. TUTORIAL TERBIMBING

1. Koneksi.py

Koneksi MongoDB berhasil!

Indikator berhasil nyambung ke MongoDB

2. Crud sensor.py

```
d for removal in a future version. Use timezone-aware objects to represent date
"timestamp": datetime.utcnow() - timedelta(minutes=i*5),
Tersimpan 10 dokumen
Data CNC-01 terbaru:
{'suhu': 73.25, 'timestamp': datetime.datetime(2026, 5, 6, 1, 1, 42, 697000)}
{'suhu': 68.38, 'timestamp': datetime.datetime(2026, 5, 6, 0, 46, 42, 697000)}
{'suhu': 73.11, 'timestamp': datetime.datetime(2026, 5, 6, 0, 41, 42, 697000)}
{'suhu': 99.23, 'timestamp': datetime.datetime(2026, 5, 5, 11, 2, 35, 982000)}
{'suhu': 79.75, 'timestamp': datetime.datetime(2026, 5, 5, 11, 2, 31, 976000)}
Update: 10 dokumen diubah
```

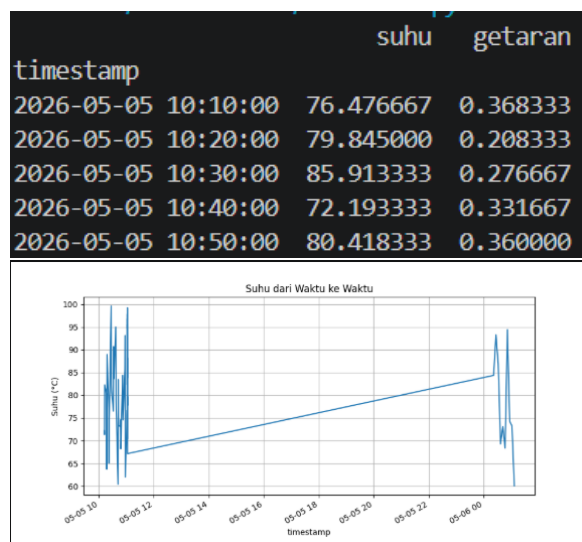
Jikalau kode di jalankan lagi, maka data akan langsung terupdate dan akan mempengaruhi pada tampilan grafiknya nanti.

3. Agregat.py

```
Rata-rata suhu per mesin:
CNC-05 -> rata:93.96°C, max getaran:0.43, jumlah data:2
CNC-03 -> rata:81.65°C, max getaran:0.48, jumlah data:22
CNC-02 -> rata:81.22°C, max getaran:0.47, jumlah data:13
CNC-04 -> rata:80.28°C, max getaran:0.35, jumlah data:5
CNC-01 -> rata:76.86°C, max getaran:0.44, jumlah data:17
```

Ini akan melakukan perhitungan jumlah data yang ada di dalam databse nya

4. Analisis.py



Bisa di lihat ketika kode dari crud.py di jalankan lagi, maka data akan langsung berubah secara acak dan terbentuklah grafik yang pasti akan berbeda beda.

5. Data_generator.py

```
print([datetime.datetime.now()]) Inserted: {doc['mesin']}
[2026-05-06 01:16:49.721204] Inserted: CNC-05 - 69.99°C
[2026-05-06 01:16:51.722411] Inserted: CNC-01 - 93.83°C
[2026-05-06 01:16:53.726092] Inserted: CNC-02 - 81.37°C
[2026-05-06 01:16:55.729177] Inserted: CNC-03 - 75.51°C
[2026-05-06 01:16:57.732384] Inserted: CNC-04 - 64.37°C
[2026-05-06 01:16:59.734480] Inserted: CNC-01 - 76.17°C
[2026-05-06 01:17:01.736062] Inserted: CNC-02 - 79.39°C
[2026-05-06 01:17:03.739152] Inserted: CNC-02 - 96.39°C
```

Ini akan terus menghasilkan data hingga kita melakukan pemberhentian paksa dengan menekan CTRL+C atau menekan stop pada terminal.

6. Mqtt Publisher dan mqtt Subscriber

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[PUB] {"mesin": "CNC-03", "suhu": 71.08, "getaran": 0.2, "timestamp": "2026-05-06T01:10:55.127909"}
[PUB] {"mesin": "CNC-02", "suhu": 82.0, "getaran": 0.45, "timestamp": "2026-05-06T01:10:57.129441"}
[PUB] {"mesin": "CNC-01", "suhu": 79.23, "getaran": 0.21, "timestamp": "2026-05-06T01:10:59.130709"}
[PUB] {"mesin": "CNC-05", "suhu": 88.02, "getaran": 0.35, "timestamp": "2026-05-06T01:11:01.132733"}
[PUB] {"mesin": "CNC-02", "suhu": 70.23, "getaran": 0.46, "timestamp": "2026-05-06T01:11:03.141173"}
[PUB] {"mesin": "CNC-01", "suhu": 82.56, "getaran": 0.12, "timestamp": "2026-05-06T01:11:05.143432"}
[PUB] {"mesin": "CNC-03", "suhu": 66.1, "getaran": 0.24, "timestamp": "2026-05-06T01:11:07.144159"}
[PUB] {"mesin": "CNC-02", "suhu": 96.25, "getaran": 0.11, "timestamp": "2026-05-06T01:11:09.144987"}
[PUB] {"mesin": "CNC-05", "suhu": 81.39, "getaran": 0.4, "timestamp": "2026-05-06T01:11:11.146489"}
[PUB] {"mesin": "CNC-03", "suhu": 83.73, "getaran": 0.33, "timestamp": "2026-05-06T01:11:13.148015"}
[PUB] {"mesin": "CNC-01", "suhu": 91.78, "getaran": 0.34, "timestamp": "2026-05-06T01:11:15.148914"}
[PUB] {"mesin": "CNC-04", "suhu": 74.18, "getaran": 0.32, "timestamp": "2026-05-06T01:11:17.149402"}
[PUB] {"mesin": "CNC-04", "suhu": 94.0, "getaran": 0.25, "timestamp": "2026-05-06T01:11:19.150617"}
[PUB] {"mesin": "CNC-05", "suhu": 74.97, "getaran": 0.4, "timestamp": "2026-05-06T01:11:21.152767"}
[PUB] {"mesin": "CNC-01", "suhu": 61.1, "getaran": 0.43, "timestamp": "2026-05-06T01:11:23.155232"}
[PUB] {"mesin": "CNC-05", "suhu": 95.96, "getaran": 0.34, "timestamp": "2026-05-06T01:11:25.156252"}
[PUB] {"mesin": "CNC-01", "suhu": 90.19, "getaran": 0.19, "timestamp": "2026-05-06T01:11:27.157461"}
[PUB] {"mesin": "CNC-01", "suhu": 85.78, "getaran": 0.26, "timestamp": "2026-05-06T01:11:29.158720"}
[PUB] {"mesin": "CNC-01", "suhu": 91.67, "getaran": 0.11, "timestamp": "2026-05-06T01:11:31.159888"}
[PUB] {"mesin": "CNC-01", "suhu": 95.42, "getaran": 0.4, "timestamp": "2026-05-06T01:11:33.161214"}
[PUB] {"mesin": "CNC-02", "suhu": 81.4, "getaran": 0.35, "timestamp": "2026-05-06T01:11:35.162482"}

& "d:\FILE ALL\FOLDER KULIAH\Mosquitto\Praktek\Python Industri\crud_sensor\mqtt_publisher.py"
✓ Saved to MongoDB ID: 69faa33953c99f3c3c03824c | CNC-01 82.56°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-03", "suhu": 66.1, "getaran": 0.24, "timestamp": "2026-05-06T01:11:07.144159"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c03824d | CNC-03 66.1°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-02", "suhu": 96.25, "getaran": 0.11, "timestamp": "2026-05-06T01:11:09.144987"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c03824e | CNC-02 96.25°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-05", "suhu": 81.39, "getaran": 0.4, "timestamp": "2026-05-06T01:11:11.146489"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c03824f | CNC-05 81.39°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-01", "suhu": 83.73, "getaran": 0.33, "timestamp": "2026-05-06T01:11:13.148015"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c038250 | CNC-01 83.73°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-03", "suhu": 91.78, "getaran": 0.34, "timestamp": "2026-05-06T01:11:15.148914"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c038251 | CNC-03 91.78°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-04", "suhu": 74.18, "getaran": 0.32, "timestamp": "2026-05-06T01:11:17.149402"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c038252 | CNC-04 74.18°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-04", "suhu": 94.0, "getaran": 0.25, "timestamp": "2026-05-06T01:11:19.150617"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c038253 | CNC-04 94.0°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-05", "suhu": 74.97, "getaran": 0.4, "timestamp": "2026-05-06T01:11:21.152767"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c038254 | CNC-05 74.97°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-01", "suhu": 61.1, "getaran": 0.43, "timestamp": "2026-05-06T01:11:23.155232"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c038255 | CNC-01 61.1°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-05", "suhu": 95.96, "getaran": 0.34, "timestamp": "2026-05-06T01:11:25.156252"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c038256 | CNC-05 95.96°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-01", "suhu": 90.19, "getaran": 0.19, "timestamp": "2026-05-06T01:11:27.157461"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c038257 | CNC-01 90.19°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-01", "suhu": 85.78, "getaran": 0.26, "timestamp": "2026-05-06T01:11:29.158720"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c038258 | CNC-01 85.78°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-01", "suhu": 91.67, "getaran": 0.11, "timestamp": "2026-05-06T01:11:31.159888"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c038259 | CNC-01 91.67°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-02", "suhu": 81.4, "getaran": 0.35, "timestamp": "2026-05-06T01:11:33.161214"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c03825a | CNC-02 81.4°C
Message on pabrik/sensor/suhu: {"mesin": "CNC-02", "suhu": 96.39, "getaran": 0.11, "timestamp": "2026-05-06T01:11:35.162482"}
✓ Saved to MongoDB ID: 69faa33953c99f3c3c03825b | CNC-02 96.39°C
```

Kedua terminal ini akan terus melakukan komunikasi satu sama lain melalui broker mosquitto.org

7. Logging

```
Data CNC-01 terbaru:
{'suhu': 82.58, 'timestamp': datetime.datetime(2026, 5, 6, 2, 19, 23, 195000)}
{'suhu': 68.03, 'timestamp': datetime.datetime(2026, 5, 6, 2, 14, 5, 369000)}
{'suhu': 81.72, 'timestamp': datetime.datetime(2026, 5, 6, 2, 14, 1, 331000)}
{'suhu': 85.29, 'timestamp': datetime.datetime(2026, 5, 6, 2, 13, 41, 277000)}
{'suhu': 76.86, 'timestamp': datetime.datetime(2026, 5, 6, 2, 13, 37, 267000)}
Update: 47 dokumen diubah
```

Secara output akan terlihat sama seperti crud tadi, bedanya adalah kode ini akan mencatat semua proses yang terjadi dan akan menjadi file log yang terpisah sehingga seperti buku catatan harian atau CCTV.

b. LATIHAN MANDIRI

- Output import maintenance

```
Data berhasil diinsert. Total 10 dokumen ditambahkan.  
ID dokumen yang diinsert: [ObjectId('69fae383dc681f410eca513a'), ObjectId('69fae383dc681f410eca513b'), ObjectId('69fae383dc681f410eca513c'), ObjectId('69fae383dc681f410eca513d'), ObjectId('69fae383dc681f410eca513e'), ObjectId('69fae383dc681f410eca513f'), ObjectId('69fae383dc681f410eca5140'), ObjectId('69fae383dc681f410eca5141'), ObjectId('69fae383dc681f410eca5142'), ObjectId('69fae383dc681f410eca5143')]
```

- Output query maintenance

```
2026-01      2310000  
2026-02      2490000  
2026-03      2385000  
2026-04      2310000  
2026-05      2325000  
  
Total keseluruhan: Rp 11,820,000  
  
=====selesai=====
```

- Output Publisher

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
Connecting to 170.106.137.233:34423 as client_id-pub-1778049354-9236 ...  
Published to subpub: {'batch': 'BATCH-014', 'mesin': 'LAS-01', 'jumlah': 127,  
'reject': 41}  
✓ Publish acknowledged (rc=0)  
📩 Publish acknowledged (mid=1)  
Published to subpub: {'batch': 'BATCH-020', 'mesin': 'LAS-01', 'jumlah': 392,  
'reject': 2}  
📩 Publish acknowledged (mid=2)  
Published to subpub: {'batch': 'BATCH-001', 'mesin': 'CNC-01', 'jumlah': 276,  
'reject': 26}  
📩 Publish acknowledged (mid=3)  
Published to subpub: {'batch': 'BATCH-003', 'mesin': 'CNC-01', 'jumlah': 240,  
'reject': 38}  
📩 Publish acknowledged (mid=4)  
Published to subpub: {'batch': 'BATCH-013', 'mesin': 'PRESS-01', 'jumlah': 424,  
'reject': 48}  
📩 Publish acknowledged (mid=5)  
Published to subpub: {'batch': 'BATCH-006', 'mesin': 'LAS-01', 'jumlah': 236,  
'reject': 39}  
📩 Publish acknowledged (mid=6)  
Published to subpub: {'batch': 'BATCH-015', 'mesin': 'CNC-02', 'jumlah': 190,  
'reject': 37}  
📩 Publish acknowledged (mid=7)  
Published to subpub: {'batch': 'BATCH-005', 'mesin': 'PRESS-01', 'jumlah': 370,  
'reject': 13}  
📩 Publish acknowledged (mid=8)  
Published to subpub: {'batch': 'BATCH-001', 'mesin': 'PRESS-01', 'jumlah': 201,  
'reject': 30}  
📩 Publish acknowledged (mid=9)  
Published to subpub: {'batch': 'BATCH-003', 'mesin': 'CNC-01', 'jumlah': 128,  
'reject': 38}  
📩 Publish acknowledged (mid=10)
```

- Output dari subscriber

```
▲Warning: Reject rate 48.04% di PRESS-01
Masuk: topic=subpub payload=b'{"batch": "BATCH-005", "mesin": "PAINT-01", "jumlah": 229, "reject": 14}'
▲Warning: Reject rate 6.11% di PAINT-01
Masuk: topic=subpub payload=b'{"batch": "BATCH-015", "mesin": "CNC-02", "jumlah": 342, "reject": 16}'
Masuk: topic=subpub payload=b'{"batch": "BATCH-007", "mesin": "CNC-02", "jumlah": 448, "reject": 42}'
▲Warning: Reject rate 9.38% di CNC-02
Masuk: topic=subpub payload=b'{"batch": "BATCH-005", "mesin": "PAINT-01", "jumlah": 379, "reject": 42}'
▲Warning: Reject rate 11.08% di PAINT-01
Masuk: topic=subpub payload=b'{"batch": "BATCH-016", "mesin": "CNC-01", "jumlah": 175, "reject": 50}'
▲Warning: Reject rate 28.57% di CNC-01
Masuk: topic=subpub payload=b'{"batch": "BATCH-008", "mesin": "CNC-02", "jumlah": 284, "reject": 47}'
▲Warning: Reject rate 16.55% di CNC-02
Masuk: topic=subpub payload=b'{"batch": "BATCH-006", "mesin": "CNC-01", "jumlah": 376, "reject": 32}'
▲Warning: Reject rate 8.51% di CNC-01
Masuk: topic=subpub payload=b'{"batch": "BATCH-002", "mesin": "PRESS-01", "jumlah": 258, "reject": 4}'
Masuk: topic=subpub payload=b'{"batch": "BATCH-014", "mesin": "LAS-01", "jumlah": 129, "reject": 36}'
▲Warning: Reject rate 27.91% di LAS-01
Masuk: topic=subpub payload=b'{"batch": "BATCH-008", "mesin": "LAS-01", "jumlah": 174, "reject": 35}'
▲Warning: Reject rate 20.11% di LAS-01
Masuk: topic=subpub payload=b'{"batch": "BATCH-011", "mesin": "PRESS-01", "jumlah": 401, "reject": 40}'
▲Warning: Reject rate 9.98% di PRESS-01
Masuk: topic=subpub payload=b'{"batch": "BATCH-015", "mesin": "PRESS-01", "jumlah": 206, "reject": 35}'
▲Warning: Reject rate 16.99% di PRESS-01
Masuk: topic=subpub payload=b'{"batch": "BATCH-016", "mesin": "CNC-02", "jumlah": 448, "reject": 42}'
```

3. Penjelasan Setiap Stage

Fase 1 (Setup): Isolasi & Aman. `venv` biar library gak bentrok, `.env` buat nyimpen "rahasia" (URI/DB) biar kode lebih rapi dan profesional.

Fase 2 (CRUD): Efisiensi Data. `insert_many` buat kirim borongan biar kenceng, `$gt` buat filter kondisi kritis, dan `$set` buat update data tanpa ngerusak field lain.

Fase 3 (Agregasi): Laporan Cepat. Ngitung rata-rata & nilai max langsung di server MongoDB biar hemat transfer data dan lebih cepet.

Fase 4 (Pandas): Analisis Tren. Resampling buat ngurangin noise data sensor, terus di-plot jadi grafik biar polanya gampang dibaca manusia.

Fase 5 & 5B (MQTT): Aliran Real-time. Simulasi pabrik beneran pake JSON biar standar, dan callback (`on_message`) biar sistem otomatis gerak pas ada data masuk.

Fase 6 (Logging): Audit & Stabilitas. Logging itu "CCTV" kodingan buat lacak error lewat file, `try-except` biar program tetep idup walau ada gangguan.

4. Analisis dan Kesimpulan

Analisis

Berdasarkan implementasi integrasi antara Python, MongoDB, dan protokol MQTT yang telah dilaksanakan, berikut adalah analisis teknis terhadap setiap komponen sistem:

Arsitektur Aliran Data (Data Pipeline): Sistem ini mendemonstrasikan jalur data end-to-end yang terisolasi menggunakan Virtual Environment untuk menjaga stabilitas dependensi proyek. Konfigurasi berbasis file `.env` memastikan bahwa parameter sensitif seperti URI database dan nama database tidak tercampur dalam kode program, sehingga meningkatkan keamanan dan fleksibilitas deployment.

Efisiensi Manajemen Database: Penggunaan metode `insert_many` pada operasi CRUD terbukti lebih efisien dibandingkan penyisipan data tunggal karena mengurangi latensi komunikasi antara aplikasi dan server MongoDB. Selain itu, penerapan operator pemfilteran seperti `$gt` (Greater Than) dan `$set` memungkinkan pembaruan status mesin secara spesifik tanpa mengganggu integritas data lainnya dalam satu koleksi.

Transformasi Data Mentah menjadi Informasi: Melalui Aggregation Pipeline, sistem mampu mengolah ribuan dokumen menjadi ringkasan statistik (rata-rata suhu dan getaran maksimum) secara langsung di sisi server. Hal ini sangat krusial dalam lingkungan industri untuk menghemat transfer data dan mempercepat proses pengambilan keputusan.

Analisis Runtun Waktu (Time Series Analysis): Integrasi dengan pustaka Pandas memungkinkan konversi data sensor menjadi format `datetime64`. Teknik resampling dalam interval tertentu (misalnya 10 menit) terbukti efektif untuk mereduksi noise dari sensor frekuensi tinggi, sehingga tren perubahan suhu dapat divisualisasikan dengan lebih jelas melalui grafik garis Matplotlib.

Komunikasi Real-Time via MQTT: Protokol MQTT menyediakan komunikasi yang ringan dan event-driven melalui mekanisme Publisher-Subscriber. Dengan menggunakan callback seperti `on_message`, sistem dapat menyimpan data ke MongoDB secara otomatis sesaat setelah data dikirimkan oleh sensor, yang mencerminkan sistem monitoring produksi secara real-time.

Resiliensi dan Pemeliharaan Sistem: Implementasi error handling menggunakan blok `try-except` pada operasi kritis (seperti koneksi database) mencegah sistem berhenti secara mendadak (crash) saat terjadi gangguan jaringan. Penggunaan pustaka logging yang mencatat kejadian ke dalam file `.log` memberikan jejak kronologis yang sangat berguna untuk proses audit dan diagnosis masalah di masa mendatang.

Kesimpulan

Praktikum ini memberikan pemahaman mendalam mengenai integrasi sistem NoSQL dengan bahasa pemrograman Python dalam konteks industri 4.0. Kesimpulan utama dari kegiatan ini adalah:

Integrasi Teknologi: Penggabungan MongoDB sebagai database dokumen, MQTT sebagai protokol komunikasi, dan Pandas sebagai alat analisis merupakan arsitektur yang sangat relevan untuk mengelola data IoT skala besar.

Optimasi Performa: Penggunaan agregasi di sisi server dan teknik resampling data runtun waktu secara signifikan meningkatkan efisiensi pemrosesan data mentah menjadi wawasan (insight) yang dapat ditindaklanjuti.

Standar Profesional: Penerapan best practice dalam pemrograman—seperti penggunaan virtual environment, pemisahan konfigurasi, penanganan kesalahan (error handling), dan sistem log—merupakan aspek fundamental dalam membangun perangkat lunak industri yang reliabel, aman, dan mudah dipelihara.

Semua file kode :

https://drive.google.com/drive/folders/1sWAcR6GGI4OKktxrwq0F_Mk4mHHt6oxj?usp=sharing